

Project Database Model and Design Report

Travelog

Team 4

Carolyna Sias (carolyna.sias@sjsu.edu), Meghana Indukuri (meghana.indukuri@sjsu.edu), Isaiah Villanueva (isaiah.villanueva@sjsu.edu)

Table of Contents

Functional Requirements -----	3
Entities, Attributes, relationships with Multiplicity -----	8
Entity-Relationship Diagram -----	15
Relational Schemas -----	16
MySQL Workbench Tables -----	17

Functional Requirements

For this application, we introduce three user views: Traveler, Company, and Administrator.

Traveler View:

Critically, on Travelog, a traveler is able to register, login, logout, and delete their account.

Register Account

Description:

A traveler can create a new account on Travelog. The system will collect registration information, validate it, and store a new record in the Users table on the database. Duplicate accounts will not be allowed via unique constraints on the username and the email account. For security purposes, the password will also be encrypted before being inputted into the database.

Input:

The registration form will have the following fields:

Username (String, Required, Unique)

Email Address (String, Required, Unique, Valid Email Format)

Password (String, Required, Encrypted Before Storage)

Process:

1. The traveler accesses the registration page and will enter the information into the form fields.
2. The system will validate that all required fields are complete
3. The system verifies that username and email address does not already exist in the Users table, the email format is valid, and that the password meets required security measures (e.g., length and alphanumeric constraints).
4. If validation passes, then the system will encrypt the password, and insert a new record into the users table containing a username, email, and encrypted password. A confirmation message will also be displayed to the user and they will be redirected to the homepage of the application.
5. If the validation fails, then the system will cancel the registration process and show the corresponding error message.

Account Sign In

Description:

A traveler can login to an existing account on Travelog. The system will authenticate the user by comparing the collected login information with the records in the Users table on the database. If the authentication is successful, then the user will be able to access the homepage via a secure session.

Input:

The login form will have the following fields:

Username (String, Required)

Password (String, Required)

Process:

1. The traveler will enter their username and password into the login fields.
2. The system will validate that all required fields are complete
3. The system will pull the matching record from the User table in the database.
4. If the corresponding record is found, the system will redirect the user to the homepage via a secure session.
5. If the corresponding record is not found, the system will display the error message and the login is cancelled.

Account Logout

Description:

A traveller can logout of their account and their secure session will be ended by the system.

Input:

Logout (Button)

Process:

1. The traveler will click the logout button.
2. The system will terminate the current session.
3. The traveler is redirected to the login page.

Delete Account

Description:

A traveller can delete their account and remove all their data and information from Travelog.

Input:

Password (String, Required)

Process:

1. Traveller will click the delete account button
2. The system will give a confirmation message where the user needs to input their password
3. The system will compare the password with records in the User table
4. If the corresponding match is found, the system will delete the record from the User table, and perform a cascading delete to remove any associated records.
5. If the match is not found (i.e., the password is incorrect), then the system will cancel deletion and show an error message.

On Travelog, a traveler is also able to view and create trips, and pick corresponding destinations and dates for each one.

View trips

Description:

A traveller can view all the trips they have created thus far on a personal dashboard. They can see their planned, booked and cancelled trips all in one place.

Input:

Secure User Session (No Manual Input)

Process:

1. The system will do an SQL query for all the trips under the current user's id in the User_Trips table.
2. If the query is empty, a message stating no trips created will be displayed.
3. If the query has results, the trips will be displayed on the dashboard for the user in descending order by status first (i.e., planned, booked, and cancelled), then by date.

Create trip

Description:

A traveller can create a new trip that will later contain all the trip details (i.e., destination, dates, accommodations, etc.)

Input:

Trip Name (String, Required)

Process:

1. Traveler will click create trip
2. Traveler will enter the trip's name
3. The system will then enter the trip as a new record in the User_Trips table.
4. The traveler will be redirected into the trip's homepage.

Select Starting and Final destinations

Description:

A traveller can select their starting origin and their destination for the trip they created.

Input:

Trip ID (No Manual Input)

Starting Location (String, Required, Pick From List or Search)

Final Destination (String, Required, Pick From List or Search)

Process:

1. Travelers will search or pick destinations. For a search, the system will do an internal query to match the search.
2. The traveler will select and confirm the origin and destination.

3. The system updates the corresponding user_trips row in the database and the changes show on the trip's homepage.

Select preferred dates

Description:

A traveller can select their preferred dates for the trip they created.

Input:

Trip ID (No Manual Input)
Start Date (Date, Required)
End Date (Date, Required)

Process:

1. The traveler picks the start and end dates using a date picker
2. The system validates that the end date is not before the start date and is after the current date.
3. The system updates the corresponding user_trips row in the database and the changes show on the trip's homepage.

On Travelog, a traveler is also able to browse and filter transport options, confirm their choices, and update transport plans to account for changes.

Browse and filter transport options

Description:

A traveller can view the different transport options like bus, plane, or train that work with the trip's date range and destination. Results can also be filtered based on certain criteria.

Input:

Trip ID (No Manual Input)
Selected Filters: Transport Type, Cost, Time (Optional)

Process:

1. The traveller searches in the search bar or selects filters.
2. The system will do a query for transport options based on the travellers search, selected filters, destination, and dates. If the user has saved preferences, it will be retrieved from the user_preferences table and applied.
3. The system will display all corresponding and filtered results.

Select transportation

Description:

A traveller can select their transportation for both trips on the journey (to and from).

Input:

Trip ID (No Manual Input)
Selected Transport (Transport ID (automatic based on selection), Required)
Direction (Dropdown, Required, Return or Outbound)

Process:

1. The traveller selects the transportation option they like and clicks confirm.
2. The system fills out a record with all the information and adds it to the trip_transport table.
3. If a transportation choice is already in the table, it will overwrite the previous record. This works as an update option for travellers.

On Travelog, a traveler is also able to browse and filter hotel options, confirm their choices, and update hotel plans to account for changes.

Browse and filter hotel options

The same logic as "Browse and filter transport options" applies here. Refer to the explanation for that functional requirements and replace transportation with hotel. Filters may be modified to number of beds, windows and other variables.

Select hotel

Description:

A traveller can select their hotel for their trip.

Input:

Trip ID (No Manual Input)

Selected Hotel (Hotel ID (automatic based on selection), Required)

Process:

1. The traveler selects a hotel and enters the specific dates.
2. The system validates the dates are feasible.
3. The system adds a record into the trip_hotels with the required information.

On Travelog, a traveler is able to confirm their booking and move their trip from a planned to booked status. They can also then cancel their booking, and move their booked status to cancelled. They may possibly receive a refund as well.

Review and Confirm Trip Booking

Description:

The traveler can view their cost summary of the trip (both hotel and transportation) and then finalize the booking.

Input:

Trip ID (No Manual Input)

Payment Method (Form, Required)

Process:

1. The system queries the Trip ID to get the correct information from the trip_transport and trip_hotels tables.
2. The costs pulled from the transportation and hotel tables will be summarized and calculated by the system.
3. The total summary will be shown to the traveller who can either continue with the booking or cancel and return to the trip homepage.
4. If they continue, the traveller selects their payment method and fills it out, here security is critical.
5. A new record will be created in the Payments table which will not contain any sensitive information but rather metadata about the traveller's payment for the booking.
6. The trip status is then updated from planned to booked.

Cancel Trip and Receive Refund

Description:

The traveller can cancel portions of the trip or the entire trip and receive a partial or full refund.

Input:

Trip ID (No Manual Input)

Cancellation Level (Dropdown, Required, Whole trip, hotel, or transportation)

Password (String, Required)

Process:

1. The traveler selects the level of which they are cancelling.
2. The system allows them to make a more detailed selection of what they are cancelling (i.e., one of the booked hotels or transportation).
3. The system calculates expected refunds, the traveller can then continue or cancel their cancellation.
4. The traveller enters their password to confirm their decision if they do cancel.
5. The system updates the status accordingly in all corresponding records (trips, trip_hotels, trip_transportations, payments).

Company View:

Critically, on Travelog companies can register, login, logout, and delete their account.

Register Account, Account Sign In, Account Signout, Delete Account

Fundamentally, there is no difference between this and the previously listed requirements for Travellers. Thus, to understand the register account, login, logout, and delete functionalities for companies refer to the traveler view above. One major change is that for an account created under a company, it will be manually audited by the Travelog personnel.

Furthermore, on Travelog, companies can post unseen offers, availability, and discounts for Travelog travelers. They can also delete any old or outdated offers.

Post Offer

Description:

Companies can post or update existing posts under their brand for Travelog users. These updates and offers could include things like discounts and new availability.

Input:

Destination (String, Required)
Name (String, Required)
Cost (String, Required)
Availabilities (String, Required)
Discounts (String, Required)

Process:

1. Company clicks on create or manage postings on the homepage.
2. Company enters modified information or new information for a posting.
3. System verifies all information is valid per constraints and expectations.
4. New postings will be added to the hotels or transportation tables by the system, updates will update existing records.
5. Confirmation is shown to the company once the posting is successfully added.

Delete Post

Description:

Companies can delete existing posts from Travelog so that users can no longer see it.

Input:

Posting ID (No Manual Input, Required)
Password (String, Required)

Process:

1. Company clicks on an existing post to delete.
2. Company clicks on delete confirmation.
3. The system prompts the company to enter a password. Validation by system occurs, failure cancels the deletion process.
4. If the company is successful in confirming deletion, the posting is removed from Travelog users except those who have already booked the accommodations or transportation.
5. The system archives records using a status column rather than a delete to avoid voiding existing users who have previously booked.

Administrator View:

Admins will have existing accounts, which they will simply use to login. Login functionality is equivalent to any regular user's experience logging in. Thus, the only important functionality these users have is confirming company accounts and handling user queries.

Confirm or Reject Company Accounts

Description:

Administrators can review to approve or deny a company account on the platform.

Input:

Selected Company (Click on Company)
Approval Choice (Accept or Deny, Required)

Process:

1. Administrator sees list of accounts awaiting approval and selects company for review.
2. The administrator chooses to deny or accept.
3. The status of the company account is changed by the system accordingly, and an email notification to the company is sent out regarding the decision.

4. The list of accounts is now reduced by one through system update of the list.

Handle User Queries

Description:

Administrators can deal with user accounts and issues sent through queries on the application.

Input:

Selected query (Required)

Administrative Action (Required)

Process:

1. Administrator selects a query in their inbox
2. The system retrieves the query from the Query table
3. The Administrator makes an action based on the query
4. The system records any changes made including updates to the database.

Entities, Attributes, Relationships with Multiplicity

For this application, we introduce 14 major entities:

Users

This entity is for storing all the information about user accounts in the application.

The primary key will be: User_ID

Attributes will be as follows:

- (1) User_ID
- (2) Username
- (3) Email
- (4) Password_Hash
- (5) TimeStamp_Created
- (6) Account_Status

Each attribute plays an important role, username will be placed under a unique constraint as well as email. The password will be stored as a hash per common privacy protocols. A timestamp for creation will also be stored for both data collection and possible benefits for long-term users. Account status will help determine if the account is active, suspended, pending (like when the user needs to confirm password via email) or deactivated.

Relationships with multiplicity:

- (1) A user can only belong to one subclass entity: Traveler, Company, or Administrator
- (2) A user can submit zero or many support queries (Support_Queries)
- (3) A user can make zero or many payments (Payments)
- (4) A user can own zero or many trips (Trips) through the Traveler subclass.

Travelers

This entity is for storing traveler-specific information. This is a subclass of the Users entity.

The primary key will be: User_ID

Attributes will be as follows:

- (1) User_ID
- (2) First_Name
- (3) Last_Name
- (4) Default_Departure_Location

Each attribute plays an important role, User_ID is the primary key and the foreign key for this entity set. First and last name are for some UI related information/personalization. Default_Departure_Location helps users spend less time picking a starting location for their trips.

Relationships with multiplicity:

- (1) A traveler has to be exactly one user (User)
- (2) A traveler can create zero or many trips (Trips)
- (3) A traveler can make zero or many payments (Payments)
- (4) A traveler can submit zero or many support queries (Support_Queries)

Companies

This entity is for storing company-specific information. This is a subclass of the Users entity.

The primary key will be: User_ID

Attributes will be as follows:

- (1) User_ID
- (2) Company_Name
- (3) Buisness_Type
- (4) Approval_Status
- (5) Contact_Phone
- (6) Verification_Date

User_ID is both the foreign key and primary key for this entity set. The company_name is important to help relay information about the company's public name. The business type will help inform about whether they provide transport, hotels, or even both. The approval status is for the review process in which the companies must be accepted onto the platform by an admin. Contact phone can help for the review process, and verification date will keep metadata about the review process as well.

Relationships with multiplicity:

- (1) A company has to be exactly one user (User)
- (2) A company can own zero or many hotel listings (Hotel_Listings)
- (3) A company can own zero or many transportation listings (Transportation_Listings)
- (4) A company can receive zero or many company approval reviews (Company_Approval_Reviews)
- (5) A company can submit zero or many support queries (Support_Queries)

Administrators

This entity is for storing administrator-specific information. This is a subclass of the Users entity.

The primary key will be: User_ID

Attributes will be as follows:

- (1) User_ID
- (2) Admin_Level

User_ID is both the foreign key and primary key for this entity set. The admin_level determines the administrative power this specific administrator holds.

Relationships with multiplicity:

- (1) An administrator has to be exactly one user (User)
- (2) An administrator can handle zero or many support queries (Support_Queries)
- (3) An administrator can create zero or many company approval reviews (Company_Approval_Reviews)

Trips

This entity is for storing all the information about trips, or in other words the user's travel plans. This is created by users.

The primary key will be: Trip_ID

Attributes will be as follows:

- (1) Trip_ID
- (2) User_ID

- (3) Trip_Name
- (4) Start_Location
- (5) End_Location
- (6) Start_Date
- (7) End_Date
- (8) Status
- (9) Timestamp_Created

Here Trip_ID is the primary key, but User_ID acts as a foreign key (as each trip belongs to a specific user). The trip_name can be any value and is not constrained by uniqueness. The start location and end location will be critical for the trip but also do not have a unique constraint. The same applies to the start and end date. Status helps determine if the trip is planned, booked, or cancelled. The timestamp created provides extra metadata about the trip.

Relationships with multiplicity:

- (1) A trip must belong to exactly one traveler (Traveler)
- (2) A trip can have zero or many trip hotels (Trip_Hotels)
- (3) A trip can have zero or many trip transportations (Trip_Transportations)
- (4) A trip can have zero or many payments (Payments)
- (5) A trip can have zero or many refunds (Refunds) via payments

Hotel_Listings

This entity is for storing all the hotel listings posted by approved company accounts or scraped from trusted providers.

The primary key will be: Hotel_ID

Attributes will be as follows:

- (1) Hotel_ID
- (2) Company_User_ID (optional)
- (3) Hotel_Name
- (4) Location
- (5) Description
- (6) Rating
- (7) Price_Per_Night
- (8) Availability_Status
- (9) Listing_Status

Hotel_ID is the primary key here used to identify each listing uniquely. Company_User_ID is a foreign key referencing the Company entity, but it is optional (as some of the listings may be scraped from trusted sources).

Hotel_Name, Location and Description are metadata for the users, alongside rating. Price per night is the price for travelers, availability status shows if there are rooms available to be booked. Listing status is for archived listings or hidden listings (easier to manage rather than deletion in case one user has them booked but another shouldn't be able to do so).

Relationships with multiplicity:

- (1) A hotel listing can belong to zero or one company (Companies)
- (2) A hotel listing can appear in zero or many trip hotels (Trip_Hotels)

Transportation_Listings

This entity is for storing all the transportation listings posted by approved company accounts or scraped from trusted providers.

The primary key will be: Transportation_ID

Attributes will be as follows:

- (1) Transportation_ID

- (2) Company_User_ID (optional)
- (3) Transport_Type
- (4) Transport_Name
- (5) Departure_Location
- (6) Arrival_Location
- (7) Departure_Time
- (8) Arrival_Time
- (9) Base_Price
- (10) Availability_Status
- (11) Listing_Status

Transportation_ID is the primary key here used to identify each listing uniquely. Company_User_ID is a foreign key referencing the Company entity, but it is optional (as some of the listings may be scraped from trusted sources).

Transport type, name, departure location, arrival location, times, and base price are metadata for the users to help make a choice. Availability status shows if there are seats for booking available. Listing status is for archived listings or hidden listings (easier to manage rather than deletion in case one user has them booked but another shouldn't be able to do so).

Relationships with multiplicity:

- (1) A transportation listing can belong to zero or one company (Companies)
- (2) A transportation listing can appear in zero or many trip transportations (Trip_Transportation)

Trip_Hotels

This entity is for storing all the hotels for a specific trip. This is a weak entity set because it depends on the Trip and Hotel_Listing entities.

The primary key will be: Trip_Hotel_Sequence_Num, Trip_ID, Hotel_ID

Attributes will be as follows:

- (1) Trip_Hotel_Sequence_Num
- (2) Trip_ID
- (3) Hotel_ID
- (4) Check_In_Date
- (5) Check_Out_Date
- (6) Booked_Price_Per_Night
- (7) Total_Cost
- (8) Booking_Status

The Trip_Hotel_Sequence_Num is a part of the primary key used to list each hotel for the trip (each trip will have a sequence of hotels that are being used). The Trip_Id is the foreign key referencing the Trip entity. Hotel_ID is also a foreign key that references the Hotel_Listings entity. The combination of these three keys forms the primary key for this entity. Check in date & checkout date are important metadata information for users who have booked or plan on booking the hotel. We also have booked prices per night as prices change over time, and what was booked prior may have had a cheaper price per night. The total cost shows how much will be spent, we store it as its own column because it includes all extra fees. Booking status helps determine if the booking is planned, booked, cancelled or even refunded.

Relationships with multiplicity:

- (1) A trip_hotel has to belong to exactly one trip (Trips)
- (2) A trip_hotel has to have exactly one hotel listing (Hotel_Listings)
- (3) A trip on the other hand can have zero or many trip hotels (Trip_Hotels)
- (4) A hotel listing on the other hand can have zero or many trip hotels (Trip_Hotels)

Trip_Transportation

This entity is for storing all the transportation for a specific trip. This is a weak entity set because it depends on the Trip and Transport_Listing entities.

The primary key will be: Trip_Transport_Sequence_Num, Trip_ID, Hotel_ID

Attributes will be as follows:

- (1) Trip_Transport_Sequence_Num
- (2) Trip_ID
- (3) Transport_ID
- (4) Direction
- (5) Price
- (6) Booking_Status

The Trip_Transport_Sequence_Num is a part of the primary key used to list each transport for the trip (each trip will have a sequence of transportations that are being used). The Trip_ID is the foreign key referencing the Trip entity. Transport_ID is also a foreign key that references the Transport_Listings entity. The combination of these three keys forms the primary key for this entity. The direction provides important information about which way (outbound or inbound) for the travelers. The price once again may change so we store it as an attribute, and the booking status helps us determine if the booking is planned, booked, cancelled or even refunded.

Relationships with multiplicity:

- (1) A trip_transport has to belong to exactly one trip (Trips)
- (2) A trip_transport has to have exactly one transport listing (Transport_Listings)
- (3) A trip on the other hand can have zero or many trip transports (Trip_Transports)
- (4) A transport listing on the other hand can have zero or many trip transports (Trip_Transports)

Payments

This entity is for storing all the information regarding payments for a specific booked trip transport or hotel.

The primary key will be: Payment_ID

The attributes are as follows:

- (1) Payment_ID
- (2) Trip_ID
- (3) User_ID
- (4) Amount
- (5) Payment_Method
- (6) Payment_Status
- (7) Payment_Timestamp

The Payment_ID is the primary key for this entity list. The Trip_ID is a foreign key referencing the Trip entity. The User_ID is a foreign key referencing the User entity table. The amount is the amount that the user pays, and the payment method highlights which type of payment they did (VISA, American Express, and beyond). The payment status helps determine the status of the payment (i.e., refunded or paid or declined). The timestamp shows when the payment occurred and helps with good metadata for payment transactions.

Relationships with multiplicity:

- (1) A payment must belong to exactly one trip (Trips)
- (2) A payment must be made by exactly one user (Users)
- (3) A trip can have zero or many payments (Payments)
- (4) A user can have zero or many payments (Payments)
- (5) A payment can have zero or many refunds (Refunds)

Refunds

This entity is for storing all the information regarding refunds for a specific payment done for a specific booking.

The primary key will be: Refund_ID

The attributes are as follows:

- (1) Refund_ID
- (2) Payment_ID
- (3) Refund_Amount
- (4) Refund_Reason
- (5) Refund_Timestamp

The refund_id is the primary key for this entity, while the payment_id is the foreign key for this entity. The refund amount shows how much of the original payment is paid back to the user, and the refund reason helps explain why for logging reasons. The timestamp is also good additional metadata for admins and the company Travelog in general.

Relationships with multiplicity:

- (1) A refund belongs to exactly one payment (Payments)
- (2) A payment can have zero or many refunds (Refunds)

Support_Queries

This entity is for storing all the information regarding refunds for a specific payment done for a specific booking.

The primary key will be: Query_ID

The attributes are as follows:

- (1) Query_ID
- (2) User_ID
- (3) Assigned_Admin_User_ID
- (4) Subject
- (5) Message_Body
- (6) Query_Status
- (7) Created_TimeStamp
- (8) Resolved_TimeStamp

Here, Query_ID is a primary key for this entity. The user_id is the foreign key which is pulled from the User table. This is because the query must be from some user account. Assigned_Admin_User_ID is the admin who is assigned to the query (which is optional) but is pulled from the Administrators table. Subject and message body highlight the information about the query. The status shows whether the query is completed, in progress, or done. The timestamps are additional metadata that we can use for the query information.

Relationships with multiplicity:

- (1) A support query must be submitted by exactly one user (User)
- (2) A support query can be assigned to 0 or 1 administrator (Administrators)
- (3) A user can submit zero or many support queries (Support_Queries)
- (4) An administrator can handle zero or many support queries (Support_Queries)

Company_Approval_Reviews

This entity is for storing all the review actions by administrators for companies.

The primary key will be: Review_ID

The attributes are as follows:

- (1) Review_ID
- (2) Company_User_ID
- (3) Assigned_Admin_User_ID
- (4) Decision
- (5) Review_Notes
- (6) Review_TimeStamp

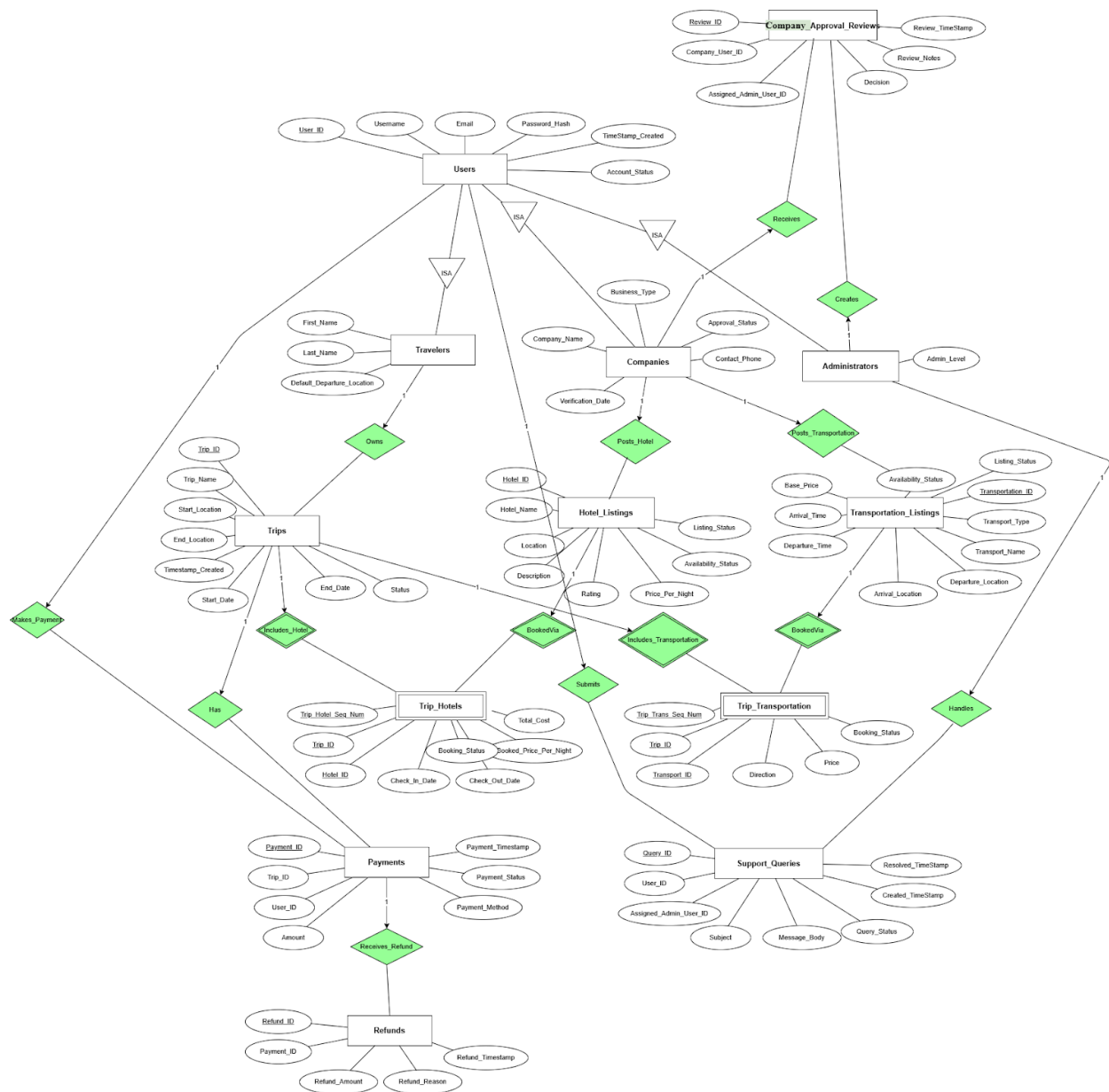
Here the primary key is the Review_ID which helps identify every unique review done. Both Company_User_ID and Assigned_Admin_User_ID are foreign keys which are pulled from the User entity. Specifically, the company is the company user to be reviewed, and the admin is the person who will review it. The decision attribute is the choice that is made on the company, and notes and timestamp give some preliminary information/metadata about why a certain decision was made.

Relationships with multiplicity:

- (1) A company approval review must reference exactly 1 company (Company)
- (2) A company approval review must be created by exactly 1 admin (Administrator)
- (3) A company can have zero or many company approval reviews (Company_Approval_Reviews)

Entity-Relationship Diagram

Link to Full Diagram: [link](#)



Relational Schemas

Entities:

User(userID, email, username, password, account_created, account_status)

Traveler(userID, FirstName, LastName, current_location)

Company(userID, name, business_type, approval_status, phone, verification_date)

Admin(userID, admin_level)

Trip(tripID, userID, trip_name, start_location, destination, start_date, end_date, status, time_created)

Hotel_Listing(hotelID, company_userID, hotel_name, location, description, rating, price_per_night, availability, listing_status)

Transport_Listing(transportID, company_userID, transport_type, transport_name, departure_location, arrival_destination, departure_time, arrival_time, base_cost, availability, listing_status)

Trip_Hotels(trip_hotel_sequence_num, tripID, hotelID, check_in_date, check_out_date, booked_price_per_night, total_cost, booking_status)

Trip_Transportation(trip_transport_sequence_num, tripID, transportID, direction, price, booking_status)

Payments(paymentID, tripID, userID, amount, payment_method, payment_status, payment_timestamp)

Refunds(refundID, paymentID, refund_amount, refund_reason, refund_timestamp)

Support_Queries(queryID, userID, assigned_admin_user_ID, subject, message_body, query_status, created_timestamp, resolved_timestamp)

Company_Approval_Reviews(reviewID, company_user_ID, admin_user_ID, decision, review_notes, review_timestamp)

Relationships:

Is_A_Traveller(User_ID)

Is_A_Company(User_ID)

Is_A_Administrator(User_ID)

Creates(User_ID, Trip_ID)

Posts_Hotel(Company_User_ID, Hotel_ID)

Posts_Transportation(Company_User_ID, Transport_ID)

Includes_Hotel(Trip_ID, Hotel_ID)

Includes_Transportation(Trip_ID, Transport_ID)

Makes_Payment(User_ID, Payment_ID)

Pays_For(Trip_ID, Payment_ID)

Receives_Refund(Payment_ID, Refund_ID)

Submits(User_ID, Query_ID)

Handles(Admin_User_ID, Query_ID)

Reviews_Company/Creates(Assigned_Admin_User_ID, Review_ID)

Reviewed_For/Receives(Company_User_ID, Review_ID)

MySQL Workbench Tables

Users:

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

Limit to 1000 rows

1 • use travelog;

2

3 • SELECT * From Users;

4

5

Result Grid

userID	email	username	password	account_created	account_status
1	meghana.indukuri@example.com	meghana_j	hashpass1	2004-01-03 08:12:00	active
2	sadhana.indukuri@example.com	sadhana_j	hashpass2	2004-01-04 09:05:00	active
3	david.baker@example.com	david_baker	hashpass3	2004-01-05 10:14:00	active
4	yann.lecun@example.com	yann_lecun	hashpass4	2004-01-06 07:48:00	active
5	ethel.cain@example.com	ethel_cain	hashpass5	2004-01-07 11:10:00	active
6	hozier.byrne@example.com	hozier	hashpass6	2004-01-08 12:26:00	active
7	andrew.ng@example.com	andrew_ng	hashpass7	2004-01-09 08:33:00	active
8	demis.hassabis@example.com	demis_h	hashpass8	2004-01-10 09:41:00	active
9	bill.gates@example.com	bill_g	hashpass9	2004-01-11 10:03:00	active
10	jeff.bezos@example.com	jeff_b	hashpass10	2004-01-12 06:57:00	active
11	marriott.biz@example.com	marriott_ops	hashpass11	2004-01-13 09:15:00	active
12	hilton.biz@example.com	hilton_team	hashpass12	2004-01-14 09:44:00	active
13	hyatt.biz@example.com	hyatt_group	hashpass13	2004-01-15 10:11:00	active
14	delta.biz@example.com	delta_routes	hashpass14	2004-01-16 11:08:00	active
15	southwest.biz@example.com	southwest...	hashpass15	2004-01-17 12:20:00	active
16	amtrak.biz@example.com	amtrak_ser...	hashpass16	2004-01-18 08:50:00	active
17	united.biz@example.com	united_travel	hashpass17	2004-01-19 07:36:00	active
18	airbnb.biz@example.com	airbnb_hos...	hashpass18	2004-01-20 09:29:00	active
19	expedia.biz@example.com	expedia_pa...	hashpass19	2004-01-21 10:47:00	active
20	booking.biz@example.com	booking_team	hashpass20	2004-01-22 11:32:00	active
21	vi.admin@example.com	vi_admin	hashpass21	2004-01-23 08:17:00	active
22	jinx.admin@example.com	jinx_admin	hashpass22	2004-01-24 09:21:00	active
23	caitlyn.admin@example.com	caitlyn_admin	hashpass23	2004-01-25 10:18:00	active
24	ekko.admin@example.com	ekko_admin	hashpass24	2004-01-26 07:59:00	active
25	jayce.admin@example.com	jayce_admin	hashpass25	2004-01-27 11:40:00	active

Users 21 x

Object Info Session

Output

Navigator: travelog - Schema Query 1 x travelog - Schema

Limit to 1000 rows

1 • use travelog;

2

3 • SELECT * From Users;

4

5

Result Grid

userID	email	username	password	account_created	account_status
7	andrew.ng@example.com	andrew_ng	hashpass7	2004-01-09 08:33:00	active
8	demis.hassabis@example.com	demis_h	hashpass8	2004-01-10 09:41:00	active
9	bill.gates@example.com	bill_g	hashpass9	2004-01-11 10:03:00	active
10	jeff.bezos@example.com	jeff_b	hashpass10	2004-01-12 06:57:00	active
11	marriott.biz@example.com	marriott_ops	hashpass11	2004-01-13 09:15:00	active
12	hilton.biz@example.com	hilton_team	hashpass12	2004-01-14 09:44:00	active
13	hyatt.biz@example.com	hyatt_group	hashpass13	2004-01-15 10:11:00	active
14	delta.biz@example.com	delta_routes	hashpass14	2004-01-16 11:08:00	active
15	southwest.biz@example.com	southwest...	hashpass15	2004-01-17 12:20:00	active
16	amtrak.biz@example.com	amtrak_ser...	hashpass16	2004-01-18 08:50:00	active
17	united.biz@example.com	united_travel	hashpass17	2004-01-19 07:36:00	active
18	airbnb.biz@example.com	airbnb_hos...	hashpass18	2004-01-20 09:29:00	active
19	expedia.biz@example.com	expedia_pa...	hashpass19	2004-01-21 10:47:00	active
20	booking.biz@example.com	booking_team	hashpass20	2004-01-22 11:32:00	active
21	vi.admin@example.com	vi_admin	hashpass21	2004-01-23 08:17:00	active
22	jinx.admin@example.com	jinx_admin	hashpass22	2004-01-24 09:21:00	active
23	caitlyn.admin@example.com	caitlyn_admin	hashpass23	2004-01-25 10:18:00	active
24	ekko.admin@example.com	ekko_admin	hashpass24	2004-01-26 07:59:00	active
25	jayce.admin@example.com	jayce_admin	hashpass25	2004-01-27 11:40:00	active
26	viktor.admin@example.com	viktor_admin	hashpass26	2004-01-28 12:02:00	active
27	mel.admin@example.com	mel_admin	hashpass27	2004-01-29 08:43:00	active
28	heimerdinger.admin@example.com	heimer_admin	hashpass28	2004-01-30 09:52:00	active
29	sevika.admin@example.com	sevika_admin	hashpass29	2004-01-31 10:24:00	active
30	vander.admin@example.com	vander_admin	hashpass30	2004-02-01 11:06:00	active

Users 21 x

Object Info Session

Output

Travelers:

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- bar_beer_drinker
- indukuri
- ticketmonarch_db

Administration Schemas

Information

No object selected

Limit to 1000 rows

```
1 • use travelog;
2
3 • SELECT * From Traveler;
4
5
```

Result Grid

	userID	firstName	lastName	current_location
1	Meghana	Indukuri	San Jose	
2	Sadhana	Indukuri	Fremont	
3	David	Baker	Seattle	
4	Yann	LeCun	New York	
5	Ethel	Cain	Nashville	
6	Hozier	Byrne	Dublin	
7	Andrew	Ng	Palo Alto	
8	Demis	Hassabis	London	
9	Bill	Gates	Redmond	
10	Jeff	Bezos	Seattle	
*	NULL	NULL	NULL	

Company:

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- bar_beer_drinker
- indukuri
- ticketmonarch_db
- travelog
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

No object selected

Limit to 1000 rows

```
1 • use travelog;
2
3 • SELECT * From Company;
4
5
```

Result Grid

	userID	name	business_type	approval_status	phone	verification_date
11	Marriott International	hotel	approved	408-555-2011	2004-02-05 09:00:00	
12	Hilton Hotels	hotel	approved	408-555-2012	2004-02-06 09:00:00	
13	Hyatt Hotels	hotel	approved	408-555-2013	2004-02-07 09:00:00	
14	Delta Air Lines	transportation	approved	408-555-2014	2004-02-08 09:00:00	
15	Southwest Airlines	transportation	approved	408-555-2015	2004-02-09 09:00:00	
16	Amtrak	transportation	approved	408-555-2016	2004-02-10 09:00:00	
17	United Airlines	transportation	approved	408-555-2017	2004-02-11 09:00:00	
18	Airbnb	hotel	approved	408-555-2018	2004-02-12 09:00:00	
19	Expedia	both	approved	408-555-2019	2004-02-13 09:00:00	
20	Booking.com	both	approved	408-555-2020	2004-02-14 09:00:00	
*	NULL	NULL	NULL	NULL	NULL	

Hotel_Listing:

The screenshot shows the MySQL Workbench interface. The top menu bar includes File, Edit, View, Query, Database, Server, Tools, Scripting, and Help. Below the menu is a toolbar with various icons. The left sidebar contains a Navigator pane with a tree view showing the database schema: travelog - Schema. Under Schemas, there are Filter objects, bar_beer_drinker, indukuri, ticketmonarch_db, and travelog. Under travelog, there are Tables, Views, Stored Procedures, and Functions. The main area displays a SQL query in the editor:

```
1 • use travelog;
2
3 • SELECT * From Hotel_Listing;
4
5
```

Below the query editor is the Result Grid pane, which shows the results of the query. It has tabs for Result Grid, Filter Rows, and others. The Result Grid tab is active, displaying a table with columns: hotelID, company_userID, hotel_name, location, description, rating, price_per_night, availability, and listing_status. The table contains 10 rows of data.

hotelID	company_userID	hotel_name	location	description	rating	price_per_night	availability	listing_status
201	11	Marriott Monterey Bay	Monterey	nice ocean view rooms and easy parking	4.5	189.00	1	active
202	12	Hilton Downtown LA	Los Angeles	close to downtown and the convention center	4.6	215.00	1	active
203	13	Hyatt Boston Harbor	Boston	quiet stay near the water	4.4	198.00	1	active
204	18	Airbnb Chicago Loft	Chicago	spacious loft with self check in	4.3	172.00	1	active
205	20	Booking Austin Suites	Austin	good for longer stays and work trips	4.2	185.00	1	active
206	19	Expedia Capitol Rooms	Washington	walkable area with breakfast included	4.5	205.00	1	active
207	11	Marriott Miami Beach	Miami	beachfront stay with pool access	4.8	255.00	1	active
208	12	Hilton Denver Center	Denver	mountain views and clean rooms	4.4	210.00	1	active
209	13	Hyatt Vegas Strip	Las Vegas	right near the main attractions	4.1	240.00	1	active
210	18	Airbnb Phoenix House	Phoenix	cozy place with patio and parking	4.0	168.00	1	active

At the bottom of the Result Grid pane, it says "No object selected".

Transport_Listing:[illegible]

Trip_Hotels

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- bar_beer_drinker
- indukuri
- ticketmonarch_db
- travelog**
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

No object selected

```

1 • use travelog;
2
3 • SELECT * From Trip_Hotels;
4
5

```

Result Grid

	trip_hotel_sequence_num	tripID	hotelID	check_in_date	check_out_date	booked_price_per_night	total_cost	booking_status
▶	1	101	201	2004-03-01	2004-03-05	189.00	756.00	planned
	1	102	202	2004-03-06	2004-03-09	215.00	645.00	booked
	1	103	203	2004-03-10	2004-03-14	198.00	792.00	planned
	1	104	204	2004-03-15	2004-03-18	172.00	516.00	booked
	1	105	205	2004-03-19	2004-03-23	185.00	740.00	cancelled
	1	106	206	2004-03-24	2004-03-27	205.00	615.00	planned
	1	107	207	2004-03-28	2004-04-01	255.00	1020.00	booked
	1	108	208	2004-04-02	2004-04-06	210.00	840.00	planned
	1	109	209	2004-04-07	2004-04-10	240.00	720.00	booked
	1	110	210	2004-04-11	2004-04-15	168.00	672.00	planned
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Trip_Transport

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- bar_beer_drinker
- indukuri
- ticketmonarch_db
- travelog**
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

No object selected

```

1 • use travelog;
2
3 • SELECT * From Trip_Transportation;
4
5

```

Result Grid

	trip_transport_sequence_num	tripID	transportID	direction	price	booking_status
▶	1	101	301	outbound	120.00	planned
	1	102	302	outbound	158.00	booked
	1	103	303	outbound	282.00	planned
	1	104	304	outbound	142.00	booked
	1	105	305	outbound	305.00	cancelled
	1	106	306	outbound	74.00	planned
	1	107	307	outbound	318.00	booked
	1	108	308	outbound	448.00	planned
	1	109	309	outbound	192.00	booked
	1	110	310	outbound	212.00	planned
*	NULL	NULL	NULL	NULL	NULL	NULL

Payments

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

Limit to 1000 rows

```

1 • use travelog;
2
3 • SELECT * From Payments;
4
5

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

paymentID	tripID	userID	amount	payment_method	payment_status	payment_timestamp
401	101	1	876.00	Visa	pending	2004-02-25 11:00:00
402	102	2	803.00	Mastercard	paid	2004-02-26 11:00:00
403	103	3	1074.00	Visa	pending	2004-02-27 11:00:00
404	104	4	658.00	PayPal	paid	2004-02-28 11:00:00
405	105	5	1045.00	Visa	refunded	2004-02-29 11:00:00
406	106	6	689.00	Debit	pending	2004-03-01 11:00:00
407	107	7	1338.00	Visa	paid	2004-03-02 11:00:00
408	108	8	1288.00	Wire Transfer	pending	2004-03-03 11:00:00
409	109	9	912.00	Amex	paid	2004-03-04 11:00:00
410	110	10	884.00	Debit	pending	2004-03-05 11:00:00
*	NULL	NULL	NULL	NULL	NULL	NULL

Administration Schemas

Information

No object selected

Refunds

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

Limit to 1000 rows

```

1 • use travelog;
2
3 • SELECT * From Refunds;
4
5

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

refundID	paymentID	refund_amount	refund_reason	refund_timestamp
501	401	30.00	small fare adjustment	2004-03-06 09:00:00
502	402	20.00	promo price correction	2004-03-07 09:00:00
503	403	45.00	service issue credit	2004-03-08 09:00:00
504	404	18.00	hotel adjustment	2004-03-09 09:00:00
505	405	1045.00	full cancellation refund	2004-03-10 09:00:00
506	406	15.00	partial refund	2004-03-11 09:00:00
507	407	55.00	delay compensation	2004-03-12 09:00:00
508	408	40.00	itinerary update refund	2004-03-13 09:00:00
509	409	25.00	billing correction	2004-03-14 09:00:00
510	410	35.00	support issued adjustment	2004-03-15 09:00:00
*	NULL	NULL	NULL	NULL

Administration Schemas

Information

No object selected

Support_Queries

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- bar_beer_drinker
- indukuri
- ticketmonarch_db
- travelog
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

No object selected

```

1 • use travelog;
2
3 • SELECT * From Support_Queries;
4
5

```

Result Grid

queryID	userID	assigned_admin_user_ID	subject	message_body	query_status	created_timestamp	resolved_timestamp
601	1	21	check in question	hi, can i check in a little early for monterey?	resolved	2004-03-01 12:00:00	2004-03-02 08:00:00
602	2	23	payment went through?	just wanted to make sure my la booking is confr...	resolved	2004-03-02 12:00:00	2004-03-03 08:00:00
603	3	25	room dates look off	my boston stay dates look a day short on the p...	in_progress	2004-03-03 12:00:00	NULL
604	4	24	train timing question	the arrival time on my chicago trip looks confusing	resolved	2004-03-04 12:00:00	2004-03-05 08:00:00
605	5	22	refund amount	can you tell me what my austin cancellation refu...	closed	2004-03-05 12:00:00	2004-03-06 08:00:00
606	6	26	bus details	is there a way to change the bus time for dc?	open	2004-03-06 12:00:00	NULL
607	7	27	hotel breakfast	does the miami hotel include breakfast or not?	in_progress	2004-03-07 12:00:00	NULL
608	8	28	flight info	i need help checking my denver itinerary again	open	2004-03-08 12:00:00	NULL
609	9	29	trip total	can someone resend my vegas trip summary?	resolved	2004-03-09 12:00:00	2004-03-10 08:00:00
610	10	30	route switch	is it possible to switch the phoenix transport opt...	open	2004-03-10 12:00:00	NULL
6039	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Company_Approval_Reviews

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- bar_beer_drinker
- indukuri
- ticketmonarch_db
- travelog
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

No object selected

```

1 • use travelog;
2
3 • SELECT * From Company_Approval_Reviews;
4
5

```

Result Grid

reviewID	company_user_ID	admin_user_ID	decision	review_notes	review_timestamp
701	11	21	approved	marriott documents checked and approved	2004-02-15 13:00:00
702	12	23	approved	hilton business profile looks good	2004-02-16 13:00:00
703	13	25	approved	hyatt verification completed	2004-02-17 13:00:00
704	14	24	approved	delta account approved after review	2004-02-18 13:00:00
705	15	22	approved	southwest submission approved	2004-02-19 13:00:00
706	16	26	approved	amtrak profile verified	2004-02-20 13:00:00
707	17	27	approved	united documents matched	2004-02-21 13:00:00
708	18	28	approved	airbnb hosting account approved	2004-02-22 13:00:00
709	19	29	approved	expedia partner account approved	2004-02-23 13:00:00
710	20	30	approved	booking account approved successfully	2004-02-24 13:00:00
NULL	NULL	NULL	NULL	NULL	NULL

Is_A_Traveller

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- bar_beer_drinker
- indukuri
- ticketmonarch_db
- travelog**
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

No object selected

Limit to 1000 rows

```

1 • use travelog;
2
3 • SELECT * From Is_A_Traveller;
4
5

```

Result Grid

userID
1
2
3
4
5
6
7
8
9
10
NULL

Is_A_Company

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- bar_beer_drinker
- indukuri
- ticketmonarch_db
- travelog**
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

No object selected

Limit to 1000 rows

```

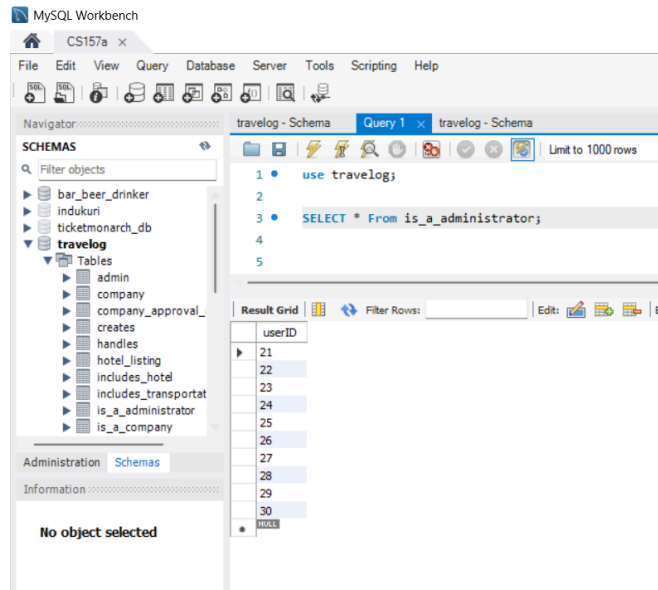
1 • use travelog;
2
3 • SELECT * From Is_A_Company;
4
5

```

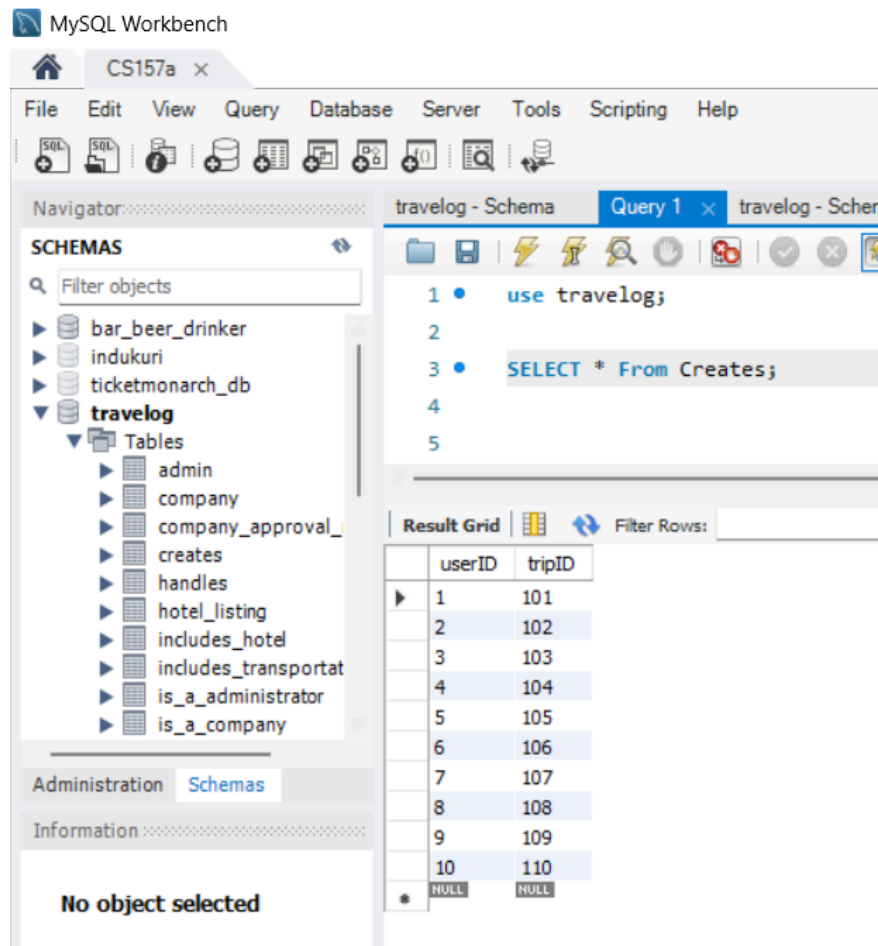
Result Grid

userID
11
12
13
14
15
16
17
18
19
20
NULL

Is_A_Administrator



Creates



Posts_Hotel

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- is_a_administrator
- is_a_company
- is_a_traveller
- makes_payment
- payments
- pays_for
- posts_hotel
- posts_transportation
- receives_refund
- refunds
- reviewed_for
- reviews_company
- submits
- support_queries
- transport_listing

Administration Schemas

Information

No object selected

1 • use travelog;

2

3 • SELECT * From Posts_Hotel;

4

5

Result Grid Filter Rows: Edit:

company_userID	hotelID
11	201
12	202
13	203
18	204
20	205
19	206
11	207
12	208
13	209
18	210
NULL	NULL

Posts_Transportation

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- is_a_administrator
- is_a_company
- is_a_traveller
- makes_payment
- payments
- pays_for
- posts_hotel
- posts_transportation
- receives_refund
- refunds
- reviewed_for
- reviews_company
- submits
- support_queries
- transport_listing

Administration Schemas

Information

No object selected

1 • use travelog;

2

3 • SELECT * From Posts_Transportation;

4

5

Result Grid Filter Rows: Edit:

company_userID	transportID
14	301
15	302
17	303
16	304
14	305
16	306
17	307
15	308
14	309
16	310
NULL	NULL

Includes_Hotel

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- is_a_administrator
- is_a_company
- is_a_traveller
- makes_payment
- payments
- pays_for
- posts_hotel
- posts_transportation
- receives_refund
- refunds
- reviewed_for
- reviews_company
- submits
- support_queries
- transport_listing

Administration Schemas

Information

No object selected

1 • use travelog;

2

3 • SELECT * From Includes_Hotel;

4

5

Result Grid

tripID	hotelID
101	201
102	202
103	203
104	204
105	205
106	206
107	207
108	208
109	209
110	210
NULL	NULL

Includes_Transportation

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- is_a_administrator
- is_a_company
- is_a_traveller
- makes_payment
- payments
- pays_for
- posts_hotel
- posts_transportation
- receives_refund
- refunds
- reviewed_for
- reviews_company
- submits
- support_queries
- transport_listing

Administration Schemas

Information

No object selected

1 • use travelog;

2

3 • SELECT * From Includes_Transportation;

4

5

Result Grid

tripID	transportID
101	301
102	302
103	303
104	304
105	305
106	306
107	307
108	308
109	309
110	310
NULL	NULL

Makes_Payment

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- is_a_administrator
- is_a_company
- is_a_traveller
- makes_payment
- payments
- pays_for
- posts_hotel
- posts_transportation
- receives_refund
- refunds
- reviewed_for
- reviews_company
- submits
- support_queries
- transport_listing

Administration Schemas

Information

No object selected

```

1 • use travelog;
2
3 • SELECT * From Makes_Payment;
4
5

```

Result Grid

	userID	paymentID
1	401	
2	402	
3	403	
4	404	
5	405	
6	406	
7	407	
8	408	
9	409	
10	410	
*	NULL	NULL

Pays_For

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- is_a_administrator
- is_a_company
- is_a_traveller
- makes_payment
- payments
- pays_for
- posts_hotel
- posts_transportation
- receives_refund
- refunds
- reviewed_for
- reviews_company
- submits
- support_queries
- transport_listing

Administration Schemas

Information

No object selected

```

1 • use travelog;
2
3 • SELECT * From Pays_For;
4
5

```

Result Grid

	tripID	paymentID
101	401	
102	402	
103	403	
104	404	
105	405	
106	406	
107	407	
108	408	
109	409	
110	410	
*	NULL	NULL

Recieves_Refund

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- is_a_administrator
- is_a_company
- is_a_traveller
- makes_payment
- payments
- pays_for
- posts_hotel
- posts_transportation
- receives_refund
- refunds
- reviewed_for
- reviews_company
- submits
- support_queries
- transport_listing

Administration Schemas

Information

Table: receives_refund

Columns:
paymentID int PK

1 • use travelog;

2

3 • SELECT * From receives_refund;

4

5

Result Grid Filter Rows: Edit:

paymentID	refundID
401	501
402	502
403	503
404	504
405	505
406	506
407	507
408	508
409	509
410	510
NULL	NULL

Submits

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Sch

SCHEMAS

Filter objects

- is_a_administrator
- is_a_company
- is_a_traveller
- makes_payment
- payments
- pays_for
- posts_hotel
- posts_transportation
- receives_refund
- refunds
- reviewed_for
- reviews_company
- submits
- support_queries
- transport_listing

Administration Schemas

Information

Table: receives_refund

Columns:
paymentID int PK

1 • use travelog;

2

3 • SELECT * From Submits;

4

5

Result Grid Filter Rows: Edit:

userID	queryID
1	601
2	602
3	603
4	604
5	605
6	606
7	607
8	608
9	609
10	610
NULL	NULL

Handles

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- is_a_administrator
- is_a_company
- is_a_traveller
- makes_payment
- payments
- pays_for
- posts_hotel
- posts_transportation
- receives_refund
- refunds
- reviewed_for
- reviews_company
- submits
- support_queries
- transport_listing

Administration Schemas

Information

Table: **receives_refund**

Columns:

admin_user_ID int(11) queryID int(11)

```

1 • use travelog;
2
3 • SELECT * From Handles;
4
5

```

Result Grid

admin_user_ID	queryID
21	601
23	602
25	603
24	604
22	605
26	606
27	607
28	608
29	609
30	610
NULL	NULL

Reviews_Company

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- is_a_administrator
- is_a_company
- is_a_traveller
- makes_payment
- payments
- pays_for
- posts_hotel
- posts_transportation
- receives_refund
- refunds
- reviewed_for
- reviews_company
- submits
- support_queries
- transport_listing

Administration Schemas

Information

Table: **reviews_company**

Columns:

admin_user_ID int(11) reviewID int(11)

```

1 • use travelog;
2
3 • SELECT * From reviews_company;
4
5

```

Result Grid

admin_user_ID	reviewID
21	701
23	702
25	703
24	704
22	705
26	706
27	707
28	708
29	709
30	710
NULL	NULL

Reviewed_For

MySQL Workbench

CS157a x

File Edit View Query Database Server Tools Scripting Help

Navigator: travelog - Schema Query 1 x travelog - Schema

SCHEMAS

Filter objects

- is_a_administrator
- is_a_company
- is_a_traveller
- makes_payment
- payments
- pays_for
- posts_hotel
- posts_transportation
- receives_refund
- refunds
- reviewed_for
- reviews_company
- submits
- support_queries
- transport_listing

Administration Schemas

Information

Table: reviews_company

Columns:

1 • use travelog;

2

3 • SELECT * From Reviewed_For;

4

5

Result Grid

	company_user_ID	reviewID
11		701
12		702
13		703
14		704
15		705
16		706
17		707
18		708
19		709
20		710
*	NULL	NULL